

Deferred readouts in the two-stage bit interchange

Draft, October 9, 2026

This note describes the round-seven bit side. It keeps the round-six interchange (flag basis, retained point totals with copied centres, side roles batched one level down, data-entrance run) and changes three things: the frames of the side program are lifted and its fan-out copies are made late, and the stage-one word is reordered so that most slots read their garbage out at a nonzero frame. We give two witnesses with two different side programs: witness 1 uses PR #62's producer (§1), witness 2, the headline, PR #57's joint frame compiler (§2); the second needs a stricter rule for the first phase of the reordered word (§5). All objects are frozen in `certificates/round7/scripts/certificate_round7.py` rebuilds both child-width histograms from the schedules, and the checks in `independent/deferred-readout/` (witness 1) and `independent/joint-frame-stack/` (witness 2) verify every fact used below. Throughout, $h = 23$, $m = h^2$, $v = \binom{h}{3}$, $N = v^2$, $G = I - J/9$, and for a triple T the root frame of the output wire y_T is $t_T^\perp = \ker(9t_T - 31)^\top$; the frame of a retained total of c is $U_c = \ker(\mathbf{1} - 3e_c)^\top$.

1 Witness 1: PR #62's producer

The side DAG has the v triples as leaves and disjoint additions as nodes; its roots are the $3v$ outputs $\text{out}(c, T) = \{S : S \cap T = \{c\}\}$ and the h retained totals $\text{star}(c)$. We build it with PR #62's producer (ikeboy): interval strips and core-aware pair assembly, with PR #41's alternating order and links (RaD/hipotures), and links chosen by a moment-weighted matching in the spirit of PR #44 (rohanarun). A link hands the non-pivot slot of a donor gate to a later use of the same value, so the number of slots is $R = \#\text{additions} + 3v + h - \#\text{links} = 28866$. Only the DAG enters the certificate, and it is checked from the definitions: every addition is disjoint, every output and total has the support above, and every node is reachable from a root (`check_lifted.py`, part A; `check_word.py`, part A).

2 Witness 2: the joint frame compiler

Witness 2 builds the side DAG with the same producer and PR #69's balanced coarse sums (eumemic; every four-edge coarse sum as two column pair sums and a total) and compiles it with PR #57's joint frame compiler (eumemic) in PR #60's rank-first reclamation order (chafreaky): roles are reused across regions of the DAG, and every gate acts at the frame of its region. The compiled program has $R = 27794$ roles. Its regions are lifted as below ($M_g = \text{span}(g) + (U_g \cap F_{j(g)})$ for a region g , with U_g the common kernel of the root covectors reachable from g along the roles), and late copies sit at exact intersections of their use frames. The compiler's region maps are $\mathbb{F}_2$ -linear, so its word identity holds over $\mathbb{F}_2$, which is all the bit network needs (§7); it is checked by an exact symbolic $\mathbb{F}_2$ replay with every scratch bit a separate symbol, for the compiled, the lifted and the final program.

3 Lifted frames and late copies

For an addition n let U_n be the common kernel of the root covectors reachable from n and F_j the span of the first j rows of a fixed integer flag matrix. The gate of n acts at

$$M_n = \text{span}(n) + (U_n \cap F_{j(n)}),$$

with a lift index $j(n)$ per addition. This is Paureel's complement-frame construction: it is valid for any frames with (H1) leaf frames $\langle t_S \rangle$, (H2) frames nested along every slot (DAG edges, links, copies), (H3) roots inside $t_T^\perp$ or U_c , (H4) G nondegenerate on every frame. Nesting is a theorem given $j(n) \leq j(t)$ on every slot successor pair, $\text{span}(x) \leq \text{span}(t)$ on every link and $\text{span}(n)$ inside every root it reaches directly; then $U_n \leq U_t$ and $M_n \leq M_t$. In this witness every lifted frame equals U_n . Dimensions are exact (modular law, ranks with Hadamard certificates or Bareiss).

Late copies. A node with several free uses is copied for each of them. A copy made early sits at M_n ; a *late* copy for the user ℓ_i of an ordered list $\ell_1, \dots, \ell_k$ is made when the pivot has climbed to

$$C_i = \bigcap_{i' \geq i} M(\ell_{i'}), \quad M_n \leq C_1 \leq \dots \leq C_{k-1} \leq M(\ell_k),$$

and starts there, so the copy saves the rank $\dim C_i - \dim M_n$. Each copy is placed as late as possible (before its user's gate, the pivot's next gate and the next copy). The intersections are needed: without them consecutive pivot

frames are not nested (1396 of 1396), and a copy moved after the pivot's next gate breaks the replay.

4 The B-defer word

In stage one each slot u starts with its garbage z_u , and the word reads it out of every target it reaches: $y_T \dashv C_{T,u} z_u$, where $C = JL$ is given by the transpose sweep of L . In rounds five and six all these readouts happen at frame 0, before the second pass. B-defer splits the second pass in two phases.

- *Phase 1* is the closure of the retained totals' last operations under per-slot sequence precedence: every op that must precede a total's completion (for witness 2 with the frame-order edges of §5). The totals complete, and their copies read every target containing their centre at 0 (copied centres, as in round six).
- A slot that phase 1 neither reads nor writes still holds its garbage after phase 1. It reads it out *now*, at a frame $\sigma_u \leq F_0(u) \cap \bigcap_T t_T^\perp$ (over its targets T), then the rest of L runs, then the output reads at $t_T^\perp$, then L^{-1} and V^{-1} exactly mirror it.

Only the topological order of the word changes, so it is still a mirror and still restores arbitrary scratch. The slot's chain now starts at σ_u : its corner has rank $r_u = h - \dim \sigma_u$ and its auxiliary edge rank $m - r_u$.

The sorting rule. On a target y_T the readout frames, in time order, are 0 (non-deferred slots and centre copies), the deferred σ_u , and $t_T^\perp$; they must form a chain. The word reads the deferred slots in increasing $\dim \sigma_u$, and applies the deferred V gates (below) in increasing $\dim F_0$. With this rule every y_T sequence is nested in time order, exactly over $\mathbb{Q}$, for both the $\mathbb{Z}$ and the $\mathbb{F}_2$ supports of C ; the reversed order breaks 11879 chains.

5 The phase-1 closure

Phase 1 must contain every op that has to run before a retained total completes. Read/write precedence alone is enough for witness 1, whose slots move along a single chain of node frames. Under the joint frame compiler a role passes through several regions, and an op that touches a role at an earlier frame of its chain must run before any op that touches it at a later

frame, or the role would have to step down its chain. The rule is therefore the closure of the retained totals' last writes under

- read/write edges: an op follows the last write of each role it reads or writes, and a write follows the earlier reads of that role;
- frame-order edges: an op touching role b at chain position k follows every op touching b at a lower position (positions from the walk of the chain in the original op order, frames compared as exact subspaces).

For witness 2 the read/write closure has 25942 ops and the full closure 31542. The checks walk the actual reordered op sequence (phase 1, centre reads, deferred readouts, deferred V gates, phase 2, output reads), and require every event to sit at a frame of both roles' chains with pointers that only move up; with the read/write closure alone that walk fails. The reordering keeps the version of every role seen by every op, so L is unchanged as a map.

6 V leaves as late copies on X_S

A leaf S is used by several gates. Instead of copying x_S by fan-out, every use i gets its own V gate on the data wire X_S , into a slot that starts at

$$s_i = N_i \cap N_{i+1} \cap \dots \quad (\text{falling back to } \langle t_S \rangle \text{ if } G \text{ is degenerate on it}),$$

where N_j is the first frame of use j . The V gates are then late copies on X_S : its frame chain $\langle t_S \rangle \leq s_{i_1} \leq s_{i_2} \leq \dots \leq F$ has total rank $h - 1$, as before, and the use slots start at s_i instead of $\langle t_S \rangle$. The V gates of deferred slots come after their readouts; in time the X_S chain is $\langle t_S \rangle$, the early V frames (by dimension), the deferred V frames (by the rule above), F , and it is checked nested exactly in that order. The number of slots is unchanged.

7 What the certificate uses

Only the $\mathbb{F}_2$ flag identity. The bit network is used over $\mathbb{F}_2$: the word must add exactly x_T to every y_T and restore every slot, for arbitrary scratch. The replay checks this over $\mathbb{F}_2$ with the parity coefficients (the actual circuit) and, separately, over $\mathbb{Z}$ with the exact path-count coefficients (witness 1); the histogram and a_b are the same with either support. For witness 2 the replay is symbolic over $\mathbb{F}_2$.

The stage-two complement. Stage two is the complement time-reversal of stage one, with the same edge multiset. This is the same assumption as in round six; it is stated, not machine-checked here.

Frames and genericity. Every σ_u and s_i is a primitive integer basis (entries at most 14). Exactly over $\mathbb{Q}$: $\sigma_u \leq F_0(u)$, $\sigma_u \leq t_T^\perp$, $t_S \leq s_i \leq N_i$, G nondegenerate. Every high-rank step ($2r > h$) that the reordered word adds (slot starts, gauged corners $\sigma_u \rightarrow F$, y_T and X_S chain steps) has the generic north-east pivots under both conjugations at the integer point of `check_lifted.py`, so the side lemma batches it.

8 Accounting

Per stage and invocation: every slot contributes its auxiliary edge (block $m - 2r_u$) and its corner, both batched by the inner lemma, and its chain steps; every centre copy its step $U_c \rightarrow 0$ of rank $h - 1$; every y_T and every X_S its chain of total rank $h - 1$; the data entrance its block $m - 4h + 2$ and corner; one copy-correction singleton per pair. The rank sum is exactly $s = Wm - N + L$ with $W = 2N + 2vR$, $L = 2vh(h - 1)$, so the histogram re-partitions the round-six budget. The moment certificate gives, with the staircase entrance corner (runs $h - 2, h - 5, 1^6$) and with the round-six corner,

$$\begin{array}{ll} \text{witness 2: } a_b = 32761/(5 \cdot 10^8), & a_b = 12899/(2 \cdot 10^8), \\ \text{witness 1: } a_b = 31987/(5 \cdot 10^8), & a_b = 12599/(2 \cdot 10^8), \end{array}$$

and with round six's complex interchange ($a_c = 36926111/(5 \cdot 10^{11})$, $h = 24$) the assembly gives the headline $\kappa = 3275885357429/(5 \cdot 10^{16}) \approx 6.55177 \times 10^{-5}$ (witness 2, staircase corner); witness 2 with the round-six corner gives $3224542033151/(5 \cdot 10^{16})$, and witness 1 gives $1599247689723/(2.5 \cdot 10^{16})$ and $6299103187973/10^{17}$. The staircase corner is certified by `check_stair.py` at the same integer point; the entrance edge is not changed by the reordering.